

Boolean conditions

Project: Python readiness portfolio

1. Your mission

Your learning target

I can translate stated requirements into comparisons.

I can combine Boolean values with and or and not.

I can choose a case that distinguishes two conditions.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Expressions and operator order. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from From a paper algorithm to Python. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Translate stated requirements into comparisons

Combine Boolean values with and or and not

Choose a case that distinguishes two conditions

Retrieve what you already know

What type is False and how does it differ from "False"?

My starting explanation

This week's end goal

Trace boolean conditions and explain the changed program before running it.

2. Notice the evidence

A fictional activity must take at most 12 minutes and be quiet. An activity taking exactly 12 minutes meets the time requirement. A quiet 15-minute activity does not. Before writing code, underline each requirement and decide whether all requirements must hold or whether alternatives are allowed.

Use a fresh `conditions.py` file with nonnegative integer minutes and an actual Boolean quiet flag. A comparison produces True or False for these values. The expression `minutes <= 12` includes 12; `minutes < 12` excludes it. Use `==` for equality and `!=` for inequality. A single `=` assigns a value to a name; it does not ask whether two values are equal.

List the given facts and one fact that is not supplied.

3. See how it works

Two requirements must both hold

Input case · Time check · Combined result

Input case: 12 min, quiet False

Time check: `12 <= 12` is True

Combined result: True and False is False

Input case: 12 min, quiet True

Time check: `12 <= 12` is True

Combined result: True and True is True

Input case: 15 min, quiet True

Time check: `15 <= 12` is False

Combined result: False and True is False

Passing only one requirement does not make the activity suitable.

```
minutes = 12
quiet = False
time_ok = minutes <= 12
suitable = time_ok and quiet
print(time_ok)
print(suitable)
```

The script prints True then False. The first result concerns time only. The second requires time_ok and quiet. These are separate claims. Trace each input before explaining the combined result; a result without its conditions can be misleading.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

For (minutes, quiet) cases (11, True), (12, False), and (13, True), write the time comparison and final suitability values before running the program.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: Change age to 16.

Prediction: The result becomes False.

Reason: The lower condition remains True, but the upper condition fails.

5. Words that unlock the idea

Word · Meaning

Word: Comparison

Meaning: An expression that checks a relation between values

Word: Boolean

Meaning: A type with the values True and False

Word: Conjunction

Meaning: A requirement that both conditions hold

Word: Inclusive or

Meaning: A condition satisfied when either or both parts hold

Word: Counterexample

Meaning: A case that shows a claim or rule is wrong

Use two terms to explain a step in this week's example.

6. Follow a worked example

A · B · A and B · A or B

A: False

B: False

A and B: False

A or B: False

A: False

B: True

A and B: False

A or B: True

A: True

B: False

A and B: False

A or B: True

A: True

B: True

A and B: True

A or B: True

With Boolean operands, and needs both parts True. Inclusive or needs at least one True, so both True is allowed. not reverses a Boolean: not True is False and not False is True. Python can also apply and and or to other types; keep operands Boolean here so this table directly describes the result.

A game requires a pencil or a crayon. If both are available, does inclusive or accept the case? Explain how this differs from requiring exactly one.

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number

it produces.

A pictorial worked case: Boolean conditions

Two conditions for entry. A toy club accepts integer ages from 12 through 15 inclusive.

The facts we will use

Python 3: `age=15; allowed=(age>=12) and (age<=15); print(allowed)`. No type validation is implemented here.

AI ACADEMY / PYTHON-BRIDGE/4

Boolean conditions

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Check lower boundary

$15 \geq 12$ is True.



2. Check upper boundary

$15 \leq 15$ is also True.



3. Combine with and

Both comparisons are True, so allowed is True.

CHANGE ONE THING

Change age to 16.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Check lower boundary

What happens and why: $15 \geq 12$ is True.

Step: 2. Check upper boundary

What happens and why: $15 \leq 15$ is also True.

Step: 3. Combine with and

What happens and why: Both comparisons are True, so allowed is True.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

A Boolean condition represents a yes-or-no result. And requires both conditions; checking only one boundary would accept values outside the intended interval.

What this example does not establish

This condition checks a declared numeric interval, not identity or authorisation; unexpected input types need separate handling.

7. Find and repair a mistake

```
minutes = 12
quiet = True
print((minutes >= 0) and (minutes <= 12))
print(not quiet)
print(minutes == 12)
```

This prints True, False and True. Parentheses make the two range checks clear. To express minutes equal to 8 or 12, write $(minutes == 8)$ or $(minutes == 12)$. Do not write $minutes == 8$ or 12 : the number 12 is not a second equality comparison. Later lessons examine truth testing of other types.

The rule uses values supplied by a person. It does not verify that the activity is really quiet, and a True result grants no permission to act. Our input contract already assumes a valid integer and Boolean; handling wrong types is a separate task.

Locate the earliest unsupported step

Complete all four Boolean combinations for pencil or crayon. Explain the both-available case and the effect

of not pencil.

State one repair and the checks you would rerun.

8. Practise together

Write a condition for an activity taking at most 12 minutes and being quiet. Explain why exactly 12 minutes is included.

A game requires a pencil or a crayon. If both are available, does inclusive or accept the case? Explain how this differs from requiring exactly one.

Explain your prediction to a partner before comparing results.

Practise with less help: Trace separate checks

Use workbook W2 for the connected example “Boolean conditions”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Use the worked case for Boolean conditions. Proposed explanation: “At most excludes the boundary”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Give both comparison results for ages 11, 15 and 16. Which comparison rejects each outside value, and why does and matter?

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

For (minutes, quiet) cases (11, True), (12, False), and (13, True), write the time comparison and final suitability values before running the program.

```
minutes = 12
quiet = False
time_ok = minutes <= 12
suitable = time_ok and quiet
print(time_ok)
```

```
print(suitable)
```

```
minutes = 12  
quiet = True  
print((minutes >= 0) and (minutes <= 12))  
print(not quiet)  
print(minutes == 12)
```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

Write a condition for an activity taking at most 12 minutes and being quiet. Explain why exactly 12 minutes is included.

Complete all four Boolean combinations for pencil or crayon. Explain the both-available case and the effect of not pencil.

Explain why a True suitability result does not prove that the supplied duration is accurate or that a child has permission to start the activity.

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled boolean conditions evidence sheet to Python readiness portfolio. Show how you can translate stated requirements into comparisons. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record: _____

Project evidence: Inputs permitted at prediction time

Record: _____

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

12. Show what you know

Testing only a quiet 8-minute activity cannot distinguish and from or because both pass. Use a case where exactly one requirement holds. Keep expected results beside observed results and explain which requirement caused failure. Next week a program will choose an action using a condition.

For the original at-most-12-minutes and quiet rule, choose one input that reveals changing $\leq$ to $<$ and a different input that reveals changing and to or. State the expected correct result for each.

Explain why a True suitability result does not prove that the supplied duration is accurate or that a child has permission to start the activity.

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

Two requirements must both hold

Input case · Time check · Combined result

Input case: 12 min, quiet False

Time check: 12 <= 12 is True

Combined result: True and False is False

Input case: 12 min, quiet True

Time check: 12 <= 12 is True

Combined result: True and True is True

Input case: 15 min, quiet True

Time check: 15 <= 12 is False

Combined result: False and True is False

Passing only one requirement does not make the activity suitable.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Combine requirements without replacing AND

Predict all four booleans. Eligibility requires science AND at most 100 pages AND availability. Repeat with genre="history" and all other values unchanged. Which OR result could wrongly suggest the genre requirement passed?

```
pages = 90

genre = "science"

available = False

print(genre == "science" and pages <= 100)

print((genre == "science" and pages <= 100) and available)

print(not available)

print(genre == "science" or pages <= 100)
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

A fictional puzzle qualifies when its duration is at least 5 and at most 9 minutes and its available flag is True. Write the condition. Predict results for (5, True), (9, True), (10, True), and (7, False). Choose a case that detects replacing and with or between the range check and available.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.